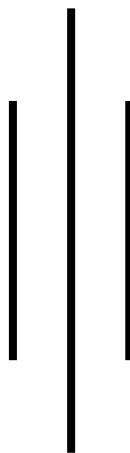


Bachelor of Science in Computer Science and Information Technology

Lab Report of Advanced JAVA Programming (CSC 406)



NAGARJUNA COLLEGE OF IT

(Affiliated to Tribhuvan University)

Banglamukhi-Sankhamul.Rd, Lalitpur

Submitted By:

Saugat Panta

Roll no: 30

Symbol No: 79010868

Semester: Seventh (7th)

Submitted To:

Er. Jeewan Rai

Lab No.	Titles	Signature
1	Write a simple java program to read from and write to files.	
2	Write a java program to process of sending email messages.	
3	Write a program using swing components to find simple interest. Use text fields for inputs and output. Your program should display the result when the user presses a button.	
4	Write a program using JavaFX components to add and subtract of two numbers. Use text fields for inputs and output. Your program should display the addition result when the user presses a button and subtract when release the button.	
5	Write a Java programs using RMI to find the addition of two numbers.	
6	Create a simple Servlet that reads and displays data from HTML form. Assume a HTML form with two fields username and password.	
7	Write TCP and UDP based socket programming using Java to communicate between client and server.	
8	Write a Java programs to insert, select, delete and update the students records (fullname, email, phoneno, gender, title) in database using JDBC	

1) Write a simple java program to read from and write to files.

Source Code:

```
import java.io.*;
import java.util.Scanner;

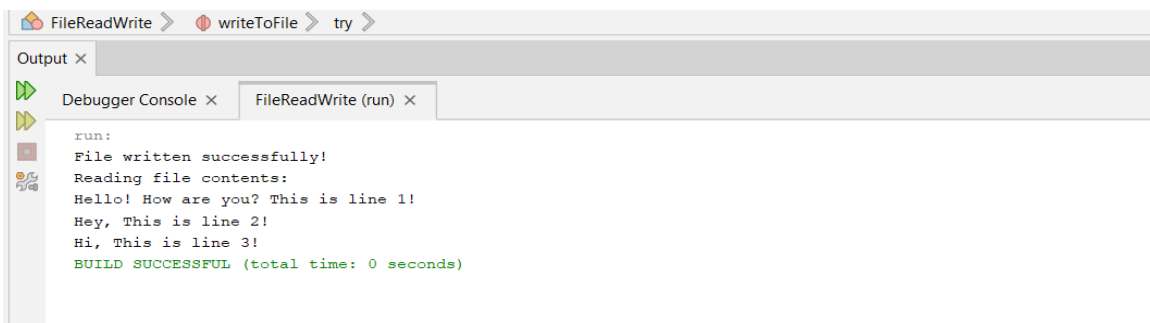
public class FileReadWrite {

    public static void main(String[] args) {
        String fileName = "myfile.txt";
        writeToFile(fileName);
        readFromFile(fileName);
    }

    public static void writeToFile(String fileName) {
        try {
            FileWriter writer = new FileWriter(fileName);
            writer.write("Hello, this is line 1!\n");
            writer.write("This is line 2!\n");
            writer.write("This is line 3!\n");
            writer.close();
            System.out.println("File written successfully!");
        } catch (IOException e) {
            System.out.println("Error writing file:");
            e.printStackTrace();
        }
    }

    public static void readFromFile(String fileName) {
        try {
            Scanner reader = new Scanner(new File(fileName));
            System.out.println("Reading file contents:");
            while (reader.hasNextLine()) {
                String line = reader.nextLine();
                System.out.println(line);
            }
            reader.close();
        } catch (FileNotFoundException e) {
            System.out.println("File not found:");
            e.printStackTrace();
        }
    }
}
```

Output:



```
FileReadWrite > writeToFile > try >
Output x
Debugger Console x  FileReadWrite (run) x
run:
File written successfully!
Reading file contents:
Hello! How are you? This is line 1!
Hey, This is line 2!
Hi, This is line 3!
BUILD SUCCESSFUL (total time: 0 seconds)
```

2) Write a java program to process of sending email messages.

Source Code:

```
import java.util.Properties;
import javax.mail.*;
import javax.mail.internet.*;

public class SendEmail {

    public static void main(String[] args) {

        final String from = "saugatpant31@gmail.com";
        final String password = "app_password_here";
        final String to = "saugatpant31@gmail.com";

        Properties props = new Properties();
        props.put("mail.smtp.host", "smtp.gmail.com");
        props.put("mail.smtp.port", "587");
        props.put("mail.smtp.auth", "true");
        props.put("mail.smtp.starttls.enable", "true");

        Session session = Session.getInstance(props,
            new Authenticator() {
                protected PasswordAuthentication getPasswordAuthentication() {
                    return new PasswordAuthentication(from, password);
                }
            });

        session.setDebug(false);

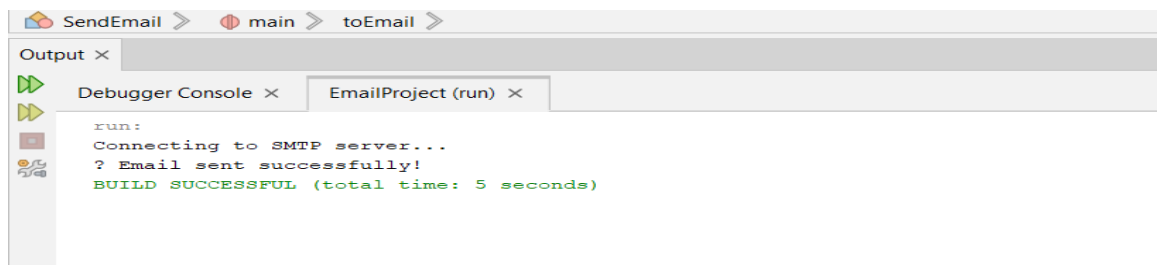
        try {
            Message message = new MimeMessage(session);
            message.setFrom(new InternetAddress(from));
            message.setRecipients(
                Message.RecipientType.TO,
                InternetAddress.parse(to)
            );
            message.setSubject("Test Email");
            message.setText("Hello, email sent successfully from Java!");

            Transport.send(message);
            System.out.println("Email sent successfully!");

        } catch (MessagingException e) {
            System.out.println("Failed to send email:");
            e.printStackTrace();
        }

    }
}
```

Output:



3) Write a program using swing components to find simple interest. Use text fields for inputs and output. Your program should display the result when the user presses a button.

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SimpleInterest extends JFrame implements ActionListener {

    JLabel principalLabel, rateLabel, timeLabel, resultLabel;
    JTextField principalField, rateField, timeField, resultField;
    JButton calculateButton, clearButton;

    public SimpleInterest() {
        setTitle("Simple Interest Calculator");
        setSize(400, 350);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(null);

        principalLabel = new JLabel("Principal Amount:");
        principalLabel.setBounds(50, 70, 150, 25);
        add(principalLabel);

        principalField = new JTextField();
        principalField.setBounds(200, 70, 150, 25);
        add(principalField);

        rateLabel = new JLabel("Rate of Interest (%):");
        rateLabel.setBounds(50, 120, 150, 25);
        add(rateLabel);

        rateField = new JTextField();
        rateField.setBounds(200, 120, 150, 25);
        add(rateField);

        timeLabel = new JLabel("Time (Years):");
        timeLabel.setBounds(50, 170, 150, 25);
        add(timeLabel);

        timeField = new JTextField();
        timeField.setBounds(200, 170, 150, 25);
        add(timeField);

        resultLabel = new JLabel("Simple Interest:");
        resultLabel.setBounds(50, 220, 150, 25);
        add(resultLabel);

        resultField = new JTextField();
        resultField.setBounds(200, 220, 150, 25);
        resultField.setEditable(false);
        add(resultField);

        calculateButton = new JButton("Calculate");
```

```

        calculateButton.setBounds(80, 275, 110, 30);
        calculateButton.addActionListener(this);
        add(calculateButton);

        clearButton = new JButton("Clear");
        clearButton.setBounds(210, 275, 110, 30);
        clearButton.addActionListener(this);
        add(clearButton);

        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == calculateButton) {
            try {
                double principal = Double.parseDouble(principalField.getText());
                double rate = Double.parseDouble(rateField.getText());
                double time = Double.parseDouble(timeField.getText());
                double si = (principal * rate * time) / 100;
                resultField.setText(String.format("%.2f", si));
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(this, "Please enter valid
numbers!");
            }
        }
        if (e.getSource() == clearButton) {
            principalField.setText("");
            rateField.setText("");
            timeField.setText("");
            resultField.setText("");
        }
    }

    public static void main(String[] args) {
        new SimpleInterest();
    }
}

```

Output:

 Simple Interest Calculator

—

□

×

Simple Interest Calculator

Principal Amount:

Rate of Interest (%):

Time (Years):

Simple Interest:

4) Write a program using JavaFX components to add and subtract of two numbers. Use text fields for inputs and output. Your program should display the addition result when the user presses a button and subtract when release the button.

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddSub extends JFrame {

    JTextField number1Field, number2Field, resultField;
    JButton operationButton;

    public AddSub() {
        setTitle("Addition and Subtraction Calculator");
        setSize(450, 320);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(null);

        JLabel num1Label = new JLabel("Number 1:");
        num1Label.setBounds(50, 70, 120, 25);
        add(num1Label);

        number1Field = new JTextField();
        number1Field.setBounds(200, 70, 180, 25);
        add(number1Field);

        JLabel num2Label = new JLabel("Number 2:");
        num2Label.setBounds(50, 120, 120, 25);
        add(num2Label);

        number2Field = new JTextField();
        number2Field.setBounds(200, 120, 180, 25);
        add(number2Field);

        JLabel resultLabel = new JLabel("Result:");
        resultLabel.setBounds(50, 170, 120, 25);
        add(resultLabel);

        resultField = new JTextField();
        resultField.setBounds(200, 170, 180, 25);
        resultField.setEditable(false);
        add(resultField);

        operationButton = new JButton("Press to Add / Release to Subtract");
        operationButton.setBounds(80, 245, 280, 35);
        add(operationButton);

        operationButton.addMouseListener(new MouseAdapter() {
            public void mousePressed(MouseEvent e) {
                double n1 = Double.parseDouble(number1Field.getText());
                double n2 = Double.parseDouble(number2Field.getText());
```

```

        resultField.setText(n1 + " + " + n2 + " = " +
            String.format("%.2f", n1 + n2));
    }
    public void mouseReleased(MouseEvent e) {
        double n1 = Double.parseDouble(number1Field.getText());
        double n2 = Double.parseDouble(number2Field.getText());
        resultField.setText(n1 + " - " + n2 + " = " +
            String.format("%.2f", n1 - n2));
    }
    });

    setVisible(true);
}

public static void main(String[] args) {
    new AddSub();
}
}

```

Output (Addition - Button Pressed):

The screenshot shows a window titled "Addition and Subtraction Calculator". Inside the window, the title "Addition and Subtraction Calculator" is displayed in a bold, dark blue font. Below the title, there are two input fields: "Number 1:" with the value "200" and "Number 2:" with the value "50". Below these fields, the "Result:" is displayed in a green box with the text "200.0 + 50.0 = 250.00". Underneath the result, the text "Press = Add Release = Subtract" is shown. At the bottom, there are two buttons: a blue button labeled "Press to Add / Release to Subtract" and a red button labeled "Clear All".

Output (Subtraction - Button Released):

Addition and Subtraction Calculator

Addition and Subtraction Calculator

Number 1:

200

Number 2:

50

Result:

200.0 - 50.0 = 150.00

Press = Add

Release = Subtract

Press to Add / Release to Subtract

Clear All

5) Write a Java programs using RMI to find the addition of two numbers.

Adder.java (Remote Interface)

```
// Adder.java (Remote Interface)
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Adder extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

AdderImpl.java (Implementation)

```
// AdderImpl.java (Server Implementation)
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class AdderImpl extends UnicastRemoteObject implements Adder {

    public AdderImpl() throws RemoteException {
        super();
    }

    public int add(int a, int b) throws RemoteException {
        return a + b;
    }
}
```

Server.java

```
// Server.java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Server {
    public static void main(String[] args) {
        try {
            AdderImpl obj = new AdderImpl();
            Registry registry = LocateRegistry.createRegistry(1099);
            registry.rebind("AdderService", obj);
            System.out.println("Server is running and ready...");
        } catch (Exception e) {
            System.out.println("Server error: " + e);
            e.printStackTrace();
        }
    }
}
```

Client.java

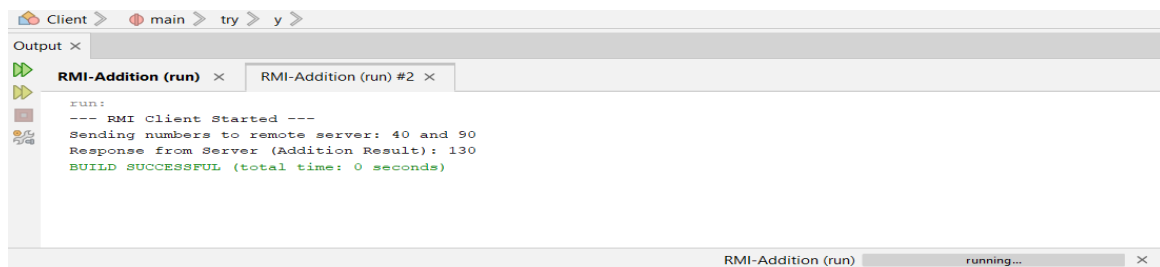
```
// Client.java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Client {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            Adder stub = (Adder) registry.lookup("AdderService");
            int result = stub.add(10, 20);
            System.out.println("Sum = " + result);
        } catch (Exception e) {
            System.out.println("Client error: " + e);
        }
    }
}
```

Server Output:



Client Output:



6) Create a simple Servlet that reads and displays data from HTML form. Assume a HTML form with two fields username and password.

HTML Form (index.html)

```
<!-- index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Login Form</title>
</head>
<body>
  <h2>Login Form</h2>
  <form action="LoginServlet" method="post">
    Username:
    <input type="text" name="username"><br><br>
    Password:
    <input type="password" name="password"><br><br>
    <input type="submit" value="Login">
  </form>
</body>
</html>
```

LoginServlet.java

```
// LoginServlet.java
package ServletFormDemo;

import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

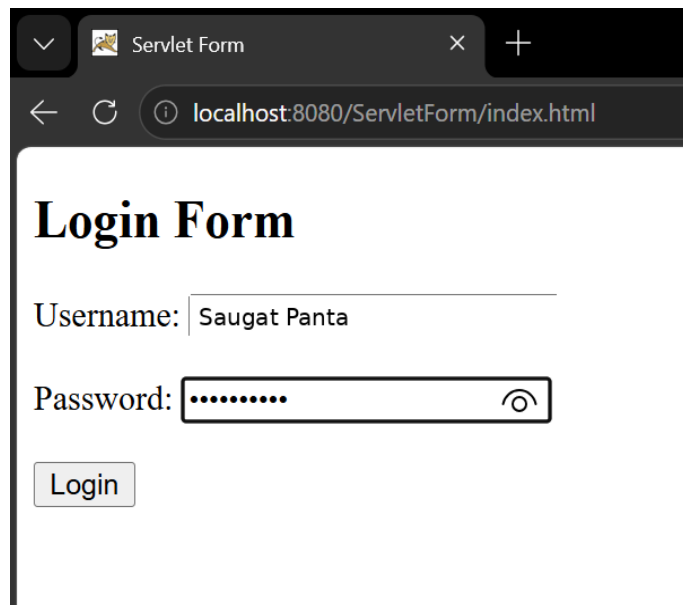
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h2>Login Details</h2>");
        out.println("<p>Username: " + username + "</p>");
        out.println("<p>Password: " + password + "</p>");

    }
}
```

Output (HTML Form):



A screenshot of a web browser window. The title bar shows a single tab labeled "Servlet Form". The address bar displays "localhost:8080/ServletForm/index.html". The page content features a large heading "Login Form". Below the heading, there are two input fields: "Username:" with the text "Saugat Panta" and "Password:" with masked characters ".....". To the right of the password field is an eye icon for toggling visibility. At the bottom left of the form is a "Login" button.

Servlet Form

localhost:8080/ServletForm/index.html

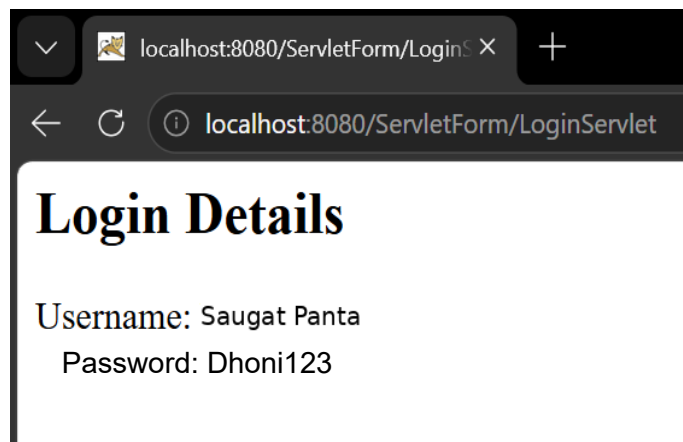
Login Form

Username: Saugat Panta

Password: 

Login

Output (Servlet Response):



A screenshot of a web browser window. The title bar shows a single tab labeled "localhost:8080/ServletForm/LoginServlet". The address bar displays "localhost:8080/ServletForm/LoginServlet". The page content features a large heading "Login Details". Below the heading, the login information is displayed: "Username: Saugat Panta" and "Password: Dhoni123".

localhost:8080/ServletForm/LoginServlet

Login Details

Username: Saugat Panta

Password: Dhoni123

7) Write TCP and UDP based socket programming using Java to communicate between client and server.

TCP Server

```
// TCPServer.java
import java.io.*;
import java.net.*;

public class TCPServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(6000);
            System.out.println("TCP Server started... Waiting for client");

            Socket socket = serverSocket.accept();
            System.out.println("Client connected!");

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

            String message = in.readLine();
            System.out.println("Client says: " + message);

            out.println("Hello from TCP Server!");

            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

TCP Client

```
// TCPClient.java
import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 6000);

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

            out.println("Hello Server, I am TCP Client!");

            String response = in.readLine();
            System.out.println("Server says: " + response);
        }
    }
}
```

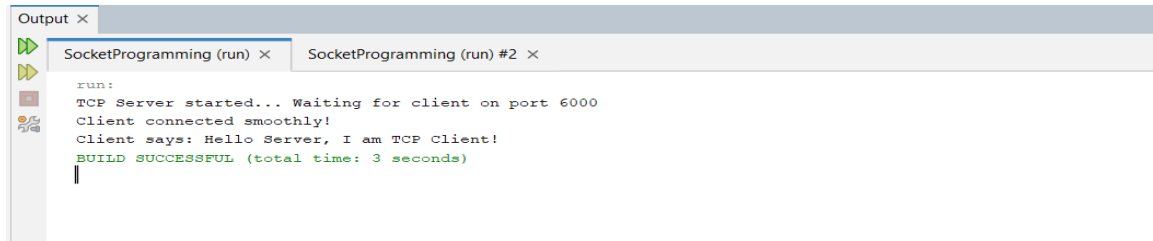


```

        socket.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

TCP Server Output:



The screenshot shows an IDE output window with two tabs: "SocketProgramming (run) ×" and "SocketProgramming (run) #2 ×". The first tab is active and displays the following text:

```

run:
TCP Server started... Waiting for client on port 6000
Client connected smoothly!
Client says: Hello Server, I am TCP Client!
BUILD SUCCESSFUL (total time: 3 seconds)

```

TCP Client Output:



The screenshot shows an IDE output window with two tabs: "SocketProgramming (run) ×" and "SocketProgramming (run) #2 ×". The second tab is active and displays the following text:

```

run:
Server says: Hello from TCP Server!
BUILD SUCCESSFUL (total time: 0 seconds)

```

UDP Server

```

// UDPServer.java
import java.net.*;

public class UDPServer {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket(5000);
            System.out.println("UDP Server started...");

            byte[] buffer = new byte[1024];
            DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
            socket.receive(packet);

            String message = new String(packet.getData(), 0, packet.getLength());
            System.out.println("Client says: " + message);

            String response = "Hello from UDP Server!";
            byte[] responseData = response.getBytes();

            DatagramPacket sendPacket = new DatagramPacket(
                responseData, responseData.length,
                packet.getAddress(), packet.getPort());

            socket.send(sendPacket);
            socket.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```
}  
}
```

UDP Client

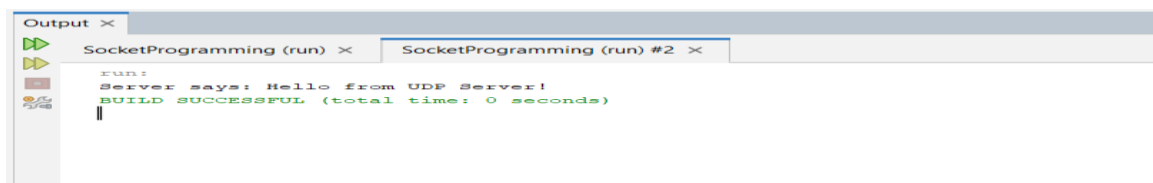
```
// UDPClient.java  
import java.net.*;  
  
public class UDPClient {  
    public static void main(String[] args) {  
        try {  
            DatagramSocket socket = new DatagramSocket();  
  
            String message = "Hello Server, I am UDP Client!";  
            byte[] data = message.getBytes();  
            InetAddress address = InetAddress.getByName("localhost");  
  
            DatagramPacket packet = new DatagramPacket(  
                data, data.length, address, 5000);  
            socket.send(packet);  
  
            byte[] buffer = new byte[1024];  
            DatagramPacket response = new DatagramPacket(buffer, buffer.length);  
            socket.receive(response);  
  
            String reply = new String(response.getData(), 0,  
response.getLength());  
            System.out.println("Server says: " + reply);  
  
            socket.close();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

UDP Server Output:



```
Output ×  
SocketProgramming (run) × SocketProgramming (run) #2 ×  
Run:  
UDP Server started on port 5000...  
Client says: Hello Server, I am UDP Client!  
BUILD SUCCESSFUL (total time: 7 seconds)
```

UDP Client Output:



```
Output ×  
SocketProgramming (run) × SocketProgramming (run) #2 ×  
Run:  
Server says: Hello from UDP Server!  
BUILD SUCCESSFUL (total time: 0 seconds)
```

8) Write a Java programs to insert, select, delete and update the students records (fullname, email, phoneno, gender, title) in database using JDBC

Source Code:

```
import java.sql.*;
import java.util.Scanner;

public class StudentCRUD {

    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "";

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("\n===== STUDENT MENU =====");
            System.out.println("1. Insert Student");
            System.out.println("2. View Students");
            System.out.println("3. Update Student");
            System.out.println("4. Delete Student");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");
            int choice = sc.nextInt();
            sc.nextLine();
            switch (choice) {
                case 1 -> insertStudent(sc);
                case 2 -> viewStudents();
                case 3 -> updateStudent(sc);
                case 4 -> deleteStudent(sc);
                case 5 -> System.exit(0);
                default -> System.out.println("Invalid choice!");
            }
        }
    }

    public static void insertStudent(Scanner sc) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
            System.out.print("Full Name: "); String name = sc.nextLine();
            System.out.print("Email: "); String email = sc.nextLine();
            System.out.print("Phone No: "); String phone = sc.nextLine();
            System.out.print("Gender: "); String gender = sc.nextLine();
            System.out.print("Title: "); String title = sc.nextLine();
            String sql = "INSERT INTO students(fullname,email,phoneno,gender,title) VALUES(?,?,?,?,?)";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setString(1, name); ps.setString(2, email);
            ps.setString(3, phone); ps.setString(4, gender); ps.setString(5, title);
            ps.executeUpdate();
            System.out.println("Student inserted successfully!");
        } catch (Exception e) { e.printStackTrace(); }
    }

    public static void viewStudents() {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
            String sql = "SELECT * FROM students";
            Statement st = con.createStatement();
        }
    }
}
```

```

        ResultSet rs = st.executeQuery(sql);
        System.out.println("\nID | Name | Email | Phone | Gender | Title");
        while (rs.next()) {
            System.out.println(
                rs.getInt("id") + " | " + rs.getString("fullname") + " | " +
                rs.getString("email") + " | " + rs.getString("phoneno") + " | " +
                rs.getString("gender") + " | " + rs.getString("title"));
        }
    } catch (Exception e) { e.printStackTrace(); }
}

public static void updateStudent(Scanner sc) {
    try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
        System.out.print("Enter Student ID to update: "); int id =
sc.nextInt(); sc.nextLine();
        System.out.print("New Phone No: "); String phone = sc.nextLine();
        System.out.print("New Title: "); String title = sc.nextLine();
        String sql = "UPDATE students SET phoneno=?, title=? WHERE id=?";
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setString(1, phone); ps.setString(2, title); ps.setInt(3, id);
        int rows = ps.executeUpdate();
        System.out.println(rows > 0 ? "Updated successfully!" : "Student not
found!");
    } catch (Exception e) { e.printStackTrace(); }
}

public static void deleteStudent(Scanner sc) {
    try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
        System.out.print("Enter Student ID to delete: "); int id =
sc.nextInt();
        String sql = "DELETE FROM students WHERE id=?";
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setInt(1, id);
        int rows = ps.executeUpdate();
        System.out.println(rows > 0 ? "Deleted successfully!" : "Student not
found!");
    } catch (Exception e) { e.printStackTrace(); }
}
}

```

Output - Insert:

```

Output - StudentCRUD (run) ...X
run:

===== STUDENT MENU =====
1. Insert Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter choice: 1
Full Name: Rameshwor Yadav
Email: yadavrameshwor123@gmail.com
Phone No: 9865327410
Gender: Male
Title: Sing and Dance
Student inserted successfully!

```

Output - Select:

```
===== STUDENT MENU =====
1. Insert Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter choice: 2

ID | Name | Email | Phone | Gender | Title
2 | Rameshwor Yadav | yadavrameshwor123@gmail.com | 9865327410 | Male | Sing and Dance

===== STUDENT MENU =====
1. Insert Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
```

Output - Update:

```
===== STUDENT MENU =====
1. Insert Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter choice: 3
Enter Student ID to update: 2
New Phone No: 9876543210
New Title: Just Sing
Updated successfully!

===== STUDENT MENU =====
1. Insert Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter choice: 2

ID | Name | Email | Phone | Gender | Title
2 | Rameshwor Yadav | yadavrameshwor123@gmail.com | 9876543210 | Male | Just Sing
```

Output - Delete:

```
===== STUDENT MENU =====
1. Insert Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter choice: 4
Enter Student ID to delete: 2
Deleted successfully!

===== STUDENT MENU =====
1. Insert Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter choice: 2

ID | Name | Email | Phone | Gender | Title
```